SimpleScheduler

Operating System : Assignment 3

Group Number: 161
Group Member: Mihir Jaiswal

Implementation

Shell Loop

The shell repeatedly displays the SimpleScheduler prompt and reads a command from the user. The user can submit an executable with the submit command and leave the shell using exit. The shell parses the command and passes valid job submissions to the scheduler.

Main functionality:

- The shell loop reads and processes user commands.
- The submit command accepts the executable path for a job.
- The exit command requests scheduler shutdown.

Job Submission and Process Creation

When a job is submitted, the scheduler creates a child process using fork(). The child initially stops itself so that it does not begin running until the scheduler gives it a time slice. The parent waits for the child to stop, records its process information, and adds the job to the ready queue. The executable is started using exec-family process execution.

Submission flow:

User enters submit ./program → shell validates command → fork() → child stops → scheduler records and queues job

Round-Robin Scheduling

The scheduler uses NCPU and TSLICE values supplied when the program starts. NCPU determines the maximum number of jobs selected for one scheduling interval, and TSLICE specifies the interval length in milliseconds. At the start of a slice, the scheduler resumes selected jobs. After the slice, it checks whether each job has finished. Jobs still running are stopped and placed at the rear of the ready queue.

Ready Queue and Synchronization

The ready queue stores jobs waiting for CPU time. Queue operations maintain the front, rear, and number of queued jobs. A mutex protects shared scheduler state, and a condition variable allows the scheduler thread to wait when no jobs are available and be notified when a job is submitted.

Job Completion and Timing

The scheduler checks child processes with waitpid(). If a process exits or is terminated by a signal, the scheduler records that the job has completed. Monotonic-clock timestamps are used to calculate elapsed time and accumulated waiting time. The final report is intended to display program name, PID, completion time in TSLICE units, and waiting time in TSLICE units.

Signal and Error Handling

The implementation checks important process and scheduling operations, including fork(), exec, kill(), waitpid(), nanosleep(), and clock_gettime(). Errors are reported where appropriate. The scheduler must handle children that exit unexpectedly and avoid waiting indefinitely for processes that no longer exist.

Limitations

This implementation is intended for the assignment requirements rather than a complete operating-system scheduler. Items to verify include:

- The shell supports the assignment submit/exit workflow rather than general shell syntax.
- Submitted jobs are expected not to use blocking input or long blocking calls such as sleep().
- The scheduler timing and waiting-time report must be checked against the assignment,expected format. The implementation has been under debugging
- Run the following tests and record the actual results before submission:
- Build using make clean && make.
- Run the scheduler with one CPU and a 500 ms time slice.
- Submit two CPU-bound test programs and verify that both complete.
- Run with NCPU greater than one and confirm that up to NCPU jobs run in each slice. Submit a short-running job and verify that completion is recorded.
- Submit an invalid executable path and check error handling.
- Exit while jobs remain and verify that shutdown handles the jobs correctly.
- Confirm the final report contains program name, PID, completion time, and waiting time.

Conclusion

SimpleScheduler is designed to demonstrate process creation, a ready queue, round-robin scheduling, time slicing, process completion detection, and timing statistics. Complete the final build and tests before submission so the implementation and report match the assignment requirements.

Group Member Contributions

Mihir Jaiswal:whole assignment

GitHub Repository

https://github.com/Mihir2024347/SimpleScheduler